

Projektarbeit Programmieren

Python – Konsolenbasiertes Wort-Ratespiel

Allgemein

Abgabe

- Einzelarbeit, keine Gruppenabgabe!
- Die Abgabe erfolgt per [Moodle](#)
- Jeder muss die Projektarbeit einzeln hochladen
- Deadline der Abgabe: 31.03.2026 - 23:59

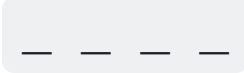
Aufgabenstellung

- Implementieren Sie ein konsolenbasiertes Wort-Ratespiel in Python.
- Zusätzlich ist eine technische Dokumentation (PDF) abzugeben.

Thematische Gestaltung

- Die Spielmechanik basiert auf einem Wort-Ratespiel mit begrenzten Fehlversuchen.
- Es ist ein **eigenes Thema** zu wählen (z.B. Codeknacken, Alien-Übersetzung, etc.).
 - Das Spiel darf **nicht** einfach "Hangman" oder "Wort-Ratespiel" heißen.
- Alle Ausgaben, Bezeichner und Texte müssen **konsistent zum gewählten Thema** passen.

Spielmechanik

- Zufälliges Wort wird verdeckt angezeigt, z.B. 
- Pro Zug:
 - Buchstabe raten
 - oder Wort direkt lösen
- 6 Fehlversuche erlaubt.
- Wiederholte Buchstaben zählen nicht als Fehler.
- Sieg: Wort vollständig erraten.
- Niederlage: 7. Fehler oder falsche Direktlösung.
- Nach Spielende: Neustart oder Beenden möglich.

Anforderungen

Technische Rahmenbedingungen

- Testumgebung: Python 3.14 unter Ubuntu 24.04
- Zulässige Module: `os`, `io`, `sys`, `random`, `unittest`
- Zulässige Tools: `pylint`, `coverage`, `mypy`

Funktionale Anforderungen

- **Wortdatenverwaltung:**

- Wörter werden aus einer Datei (`wordrepo.txt`) geladen.
- Das Spiel darf nicht in diese Datei schreiben.
- Wörter müssen validiert werden.
- Ein Wort darf nur einmal pro Programmlauf gespielt werden.

- **Spielstand-Anzeige:**

- Aktuelles Wort mit aufgedeckten, verdeckten und falschen Buchstaben.
- Bereits getestete Buchstaben und verbleibende falsche Versuche.

- **Spiellogik**

- Validierung von Buchstaben- und Wort-Eingaben.
- Aktualisierung des Spielzustands.
- Erkennung von Sieg und Niederlage.

Nicht-funktionale Anforderungen

- Keine Abstürze bei ungültiger Eingabe.
- Verständliche und klare Konsolenausgaben.

Robuste Software-Entwicklung

- Es müssen automatisierte Unittests entwickelt werden.
- Es muss die Bibliothek `unittest` genutzt werden.
- Pro Datei muss eine Testabdeckung von 75% erreicht werden (Prüfung über `coverage`).
- Die Tests müssen logisch sinnvoll entwickelt sein.

Statische Code-Analyse - Pylint

- Alle `pylint` Warnungen müssen für **jede** Datei behoben werden.
- Meldungen können durch eine sehr gute Begründung unterdrückt werden.
- In Testdateien dürfen fehlende Docstrings deaktiviert werden.
- Maximal 3 individuelle Anpassungen in `.pylintrc` sind erlaubt (mit Begründung).

Type Checker - MyPy

- Alle Python `source` Dateien müssen von mypy überprüft werden.
- Es müssen alle mypy Warnungen behoben werden.
- Es **muss** folgende `mypy.ini` Datei genutzt werden

```
[mypy]
exclude = tests
warn_return_any = True
warn_unused_configs = True
disallow_untyped_defs = True
disallow_untyped_calls = True
disallow_incomplete_defs = True
```

Technische Anforderungen

- Alle Bibliotheken müssen in der Datei `requirements.txt` verwaltet werden.
- Die Ordnerstruktur des Projektes **muss** folgendermaßen aussehen:

```
.
├── mypy.ini
├── README.md
├── requirements.txt
├── documentation
│   └── documentation.pdf
├── source
│   ├── game.py
│   ├── wordrepo.txt
│   └── <other_files.py>
└── tests
    ├── test_game.py
    └── <test_other_files.py>
```

Dokumentation (PDF)

Enthalten sein muss:

1. Beschreibung der Spielfunktionen
2. Architekturbeschreibung
 - Projektstruktur
 - Hauptmodule
 - Zusammenspiel
3. Beschreibung des Programmablaufs
4. Beschreibung der Benutzerinteraktion
5. Ergebnisse von Tests, Coverage, Pylint, MyPy
6. Versionsangaben verwendeter Bibliotheken

Einsatz von KI-Tools

- KI-Tools dürfen verwendet werden. Kenntlichmachung gemäß [DHBW Hinweisblatt](#).
- Die Verantwortung für Korrektheit, Funktionalität und Verständnis liegt bei Ihnen.
- Bewertet werden technische Qualität, Robustheit, Tests und Konsistenz – nicht die sprachliche Formulierung.
- Code, Tests, README und Dokumentation müssen inhaltlich konsistent sein.
- Inkonsistenzen führen zu Punktabzug.
- Es werden definierte Testfälle manuell geprüft.

Bewertung

- Die Bewertung erfolgt über eine Excel-Tabelle mit Selbsteinschätzung.

Kategorie	Gewichtung
Funktionsumfang (Alle Anforderungen eingehalten)	20%
Unittests + Abdeckung (Coverage + Tests)	20%
Statische Codeanalyse & Type Checker	12%
Code-Klarheit, Struktur & Konsistenz	20%
Dokumentation	28%